

Processo Seletivo – Desafio Técnico

Desenvolvedor Backend Java

1. Objetivo

Este desafio tem como objetivo avaliar:

- Domínio de fundamentos de **Java e Orientação a Objetos**
- Capacidade de **modelagem de dados relacional**
- Uso correto de **Spring Boot e JPA/Hibernate**
- Implementação de **API REST**
- Organização de código (Clean Code)
- Estruturação de regras de negócio
- Boas práticas básicas de versionamento e documentação

O foco está no raciocínio técnico e na clareza da solução, não na complexidade excessiva.

2. Contexto do Problema

Uma empresa do setor de alimentação deseja informatizar o controle de pedidos realizados em seu restaurante.

O sistema deverá permitir o gerenciamento de clientes, produtos e pedidos, garantindo regras de negócio específicas para cálculo de valores e controle de estados do pedido.

3. Escopo do Sistema

Desenvolver uma API REST para gerenciamento de pedidos de restaurante.

4. Requisitos Funcionais

4.1 Cadastro de Cliente

- Criar cliente
- Cliente possui:
 - Nome
 - Email
 - Indicação se é VIP (boolean)

4.2 Cadastro de Produto

- Criar produto
- Produto possui:
 - Nome
 - Preço
 - Ativo (boolean)

Não deve ser possível adicionar ao pedido produtos inativos.

4.3 Criação de Pedido

- Criar pedido associado a um cliente
- Pedido inicia com status: **ABERTO**
- Pedido deve registrar:
 - Data de criação
 - Status
 - Lista de itens

4.4 Adição de Itens ao Pedido

Cada item deve conter:

- Produto
- Quantidade
- Valor unitário no momento da compra

4.5 Cálculo do Valor Total

O valor total do pedido deve:

- Somar todos os itens
- Aplicar regras de desconto:

Condição	Desconto
Valor total > R\$ 200	10%
Cliente VIP	5%

Se ambas condições forem verdadeiras, os descontos são cumulativos.

4.6 Fechamento do Pedido

- Pedido pode ser fechado.
- Após fechado:
 - Não pode adicionar itens
 - Não pode alterar dados
 - Status deve mudar para **FECHADO**

5. Requisitos Técnicos

A solução deve obrigatoriamente utilizar:

- Java 17+
- Spring Boot
- JPA/Hibernate
- Banco relacional (PostgreSQL ou MySQL)
- API REST estruturada

6. Requisitos Não Funcionais

A solução será avaliada considerando:

6.1 Organização

- Separação em camadas (Controller, Service, Repository)
- Não expor entidades diretamente na API (uso de DTO é desejável)

6.2 Modelagem

- Relacionamentos corretamente definidos
- Uso adequado de **@OneToMany**, **@ManyToOne**
- Chaves estrangeiras consistentes

6.3 Clean Code

- Métodos coesos
- Nomes claros
- Baixo acoplamento
- Evitar lógica de negócio no controller

6.4 Tratamento de Erros

- Retorno adequado de status HTTP
- Mensagens claras para erros de regra de negócio

6.5 Versionamento

- Projeto versionado com Git
- Commits organizados

7. Diferenciais (Não Obrigatórios)

- Testes unitários
- Dockerfile ou docker-compose
- Documentação via Swagger/OpenAPI
- Uso de migrations (Flyway ou Liquibase)

8. Entrega

O candidato deverá entregar:

- Repositório Git (GitHub/GitLab)
- README contendo:
 - Instruções para execução
 - Tecnologias utilizadas
 - Decisões técnicas tomadas

11. Observações Importantes

- É permitido consultar documentação oficial:
 - <https://spring.io>
 - <https://docs.spring.io>
 - <https://baeldung.org>
 - etc.
- É permitido realizar pesquisas técnicas, como Stack Overflow, por exemplo.
- O uso de ferramentas de **geração automática de código ou inteligência artificial** não é permitido.

- A solução deverá ser defendida oralmente, com explicação das decisões técnicas adotadas.
- **Pausas:** Permitida pausa única de 5 a 10 minutos, se necessário.

12. Observações finais

- Simplicidade é mais importante que complexidade
- O foco é **backend** e **modelagem**
- Não existe uma solução única

Boa sorte!!!